07/09/2026

ASSIGNMENT - 01

01. Explain the concept of Feedback in a neural network. With a neat signal-flow graph, explain the single-loop feedback system and derive its input-output relationship and closed-loop operator.

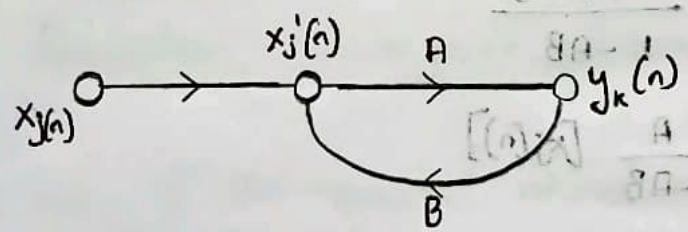
Ans:

Feedback :-

Feedback is said to exist in a dynamic system when the output of an element influences, wholly or partly, the input applied to that element, thereby forming one or more closed paths for transmission of signals around the system.

Single-Loop Feedback System :-

The system has a forward path and a feedback path. The output of the forward path is fed back through operator B, thereby influencing the input of the forward path.



where,

$x_j(n)$ → input signal

$x_j'(n)$ → Internal signal

$y_k(n)$ → Output signal

A → Forward-path operator

B → Feedback-path operator

n → Discrete-time variable

Input-Output Relationship:

→ Forward Path:

$$y_k(n) = A[x_j'(n)] \quad \rightarrow (1)$$

→ Feedback Path:

$$x_j'(n) = x_j(n) + B[y_k(n)] \quad \rightarrow (2)$$

Consider, eq^n (1)

$$y_k(n) = A[x_j'(n)]$$

Substitute eq^n (2) in eq^n (1)

$$y_k(n) = A[x_j(n) + B[y_k(n)]]$$

$$y_k(n) = A[x_j(n)] + AB[y_k(n)]$$

$$y_k(n) - AB[y_k(n)] = A[x_j(n)]$$

$$y_k(n)[1 - AB] = A[x_j(n)]$$

$$y_k(n) = \frac{A[x_j(n)]}{1 - AB}$$

$$y_k(n) = \frac{A}{1 - AB}[x_j(n)]$$

where

$\frac{A}{1-AB}$ → closed-loop operator

AB → open-loop operator.

and $BA \neq AB$

$A = w$ → fixed weight

$B \rightarrow z^{-1}$ unit delay operator

$$\Rightarrow \frac{A}{1-AB} = \frac{W}{1-Wz^{-1}}$$

$$= W(1-Wz^{-1})^{-1}$$

⇒ Using Binomial expansion for $(1-Wz^{-1})^{-1}$

$$\frac{A}{1-AB} = W\sum_{l=0}^{\infty} W^l z^{-l}$$

$$y_k(n) = W\sum_{l=0}^{\infty} W^l z^{-l} \cdot [x_j(n)]$$

$$z^{-l}[x_j(n)] = x_j(n-l)$$

$$y_k(n) = \sum_{l=0}^{\infty} W^{l+1} x_j(n-l)$$

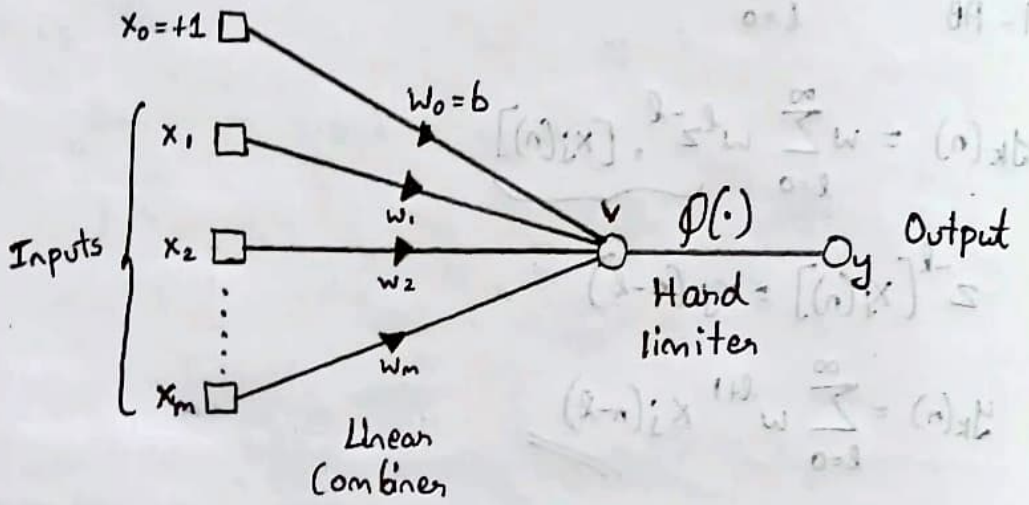
Therefore Output signal, $y_k(n) = \sum_{l=0}^{\infty} W^{l+1} x_j(n-l)$

02. Explain the Perceptron Convergence Theorem.

Ans: Perceptron: Is one of the simplest neural-network models used for classification of linearly separable patterns. It consists of a single neuron with adjustable synaptic weights and a bias.

Rosenblatt's Perceptron Convergence Theorem states that:

If the patterns used to train a perceptron are drawn two linearly separable classes, then the perceptron learning algorithm converges and positions the decision surface as a hyperplane between the two classes.



where

$x(n) = [+1, x_1, x_2, \ldots, x_m]$ → Input Signals

$w(n) = [b(n), w_1, w_2 \ldots w_m]$ → Synaptic weights.

$b(n)$ → Bias.

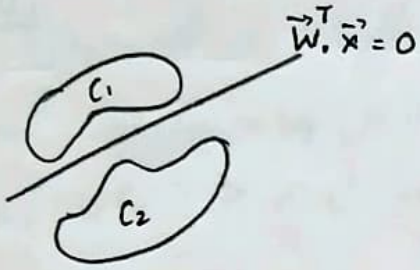
$\phi(\cdot)$ → Hard-limiter activation function.

$$v(n) = \sum_{i=0}^{m} w_i(n)\, x_i(n)$$

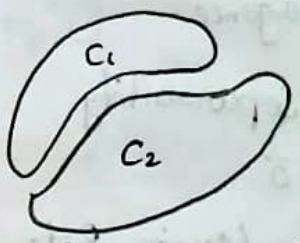
$$v(n) = \vec{w}^T(n) \cdot \vec{x}(n)$$

→ 2-class Pattern Classification:

C_1, C_2 : Linearly separable Class.



(a) Linearly Separable problems



(b) Non-Linearly Separable problems.

→ H : Training Set

$H : H_1 \cup H_2$

$H_1 \in C_1$ $\quad H_1 \subset C_1$

$H_2 \in C_2$ $\quad H_2 \subset C_2$

→ Rules:

(i) $\vec{w}^T \cdot \vec{x} > 0$ for all input vector $x \in C_1$

(ii) $\vec{w}^T \cdot \vec{x} \leq 0$ for all input vector $x \in C_2$

→ If right → No change in w

If wrong → update the weights

⇒ Update Algorithms:

(1) $\vec{w}(n+1) = \vec{w}(n)$ if $w^T x(n) > 0$ and $x(n) \in C_1$

(2) $\vec{w}(n+1) = \vec{w}(n)$ if $w^T x(n) \leq 0$ and $x(n) \in C_2$

(2) $\vec{w}(n+1) = w(n) - \eta\, x(n)$ if $w^T(n) \cdot x(n) > 0$ and $x(n) \in C_2$

$\vec{w}(n+1) = w(n) + \eta\, x(n)$ if $w^T(n) \cdot x(n) \leq 0$ and $x(n) \in C_1$

* Proof of Convergence:

* Proof of Convergence:

① Linear separability

② $\vec{w}(0) = \vec{0}$

③ $\eta = 1$ → Learning Rate

→ Given:

$\vec{x}(n)$. , $n = 1, 2, 3 \cdots$

→ Proof:

Let $\vec{x}(n) \in H_1$, $n = 1$, for $x(n) \in C_1$

$\vec{w}(n+1) = \vec{w}(n) + \eta \vec{x}(n)$

$\vec{w}(n+1) = \vec{w}(n) + \vec{x}(n)$ → for $\eta = 1$ → (A)

for $n = 0$

$\vec{w}(1) = \vec{w}(0) + 1\,\vec{x}(0)$

$\vec{w}(1) = 0 + \vec{x}(0)$

$\vec{w}(1) = \vec{x}(0)$

for $n = 1$

$\vec{w}(2) = \vec{w}(1) + \vec{x}(1)$

$\vec{w}(2) = \vec{x}(0) + \vec{x}(1)$

⋮

for $n = n$

$\vec{w}(n+1) = \vec{w}(n) + \vec{x}(n)$

$\vec{w}(n+1) = \vec{x}(0) + \vec{x}(1) + \cdots\cdots + \vec{x}(n)$ ⟶ ①

→ C_1 and C_2 are Linearly Separable

A solution $\vec{w}_0$ exist for which

$\vec{w}_0^T \vec{x}(n) > 0$ for $\vec{x}(1), \vec{x}(2) \cdots\cdots \vec{x}(n) \in H_1$

$\alpha = \min_{\vec{x}(n) \in H_1} \vec{w}_0^T \vec{x}(n)$ → ②

where α => Positive number

multiply w_0^T on both sides of eq^ ①

$$w_0^T\, w(n+1) = w_0^T(x(1)) + w_0^T \cdot x(2) + \dots + w_0^T \cdot x(n)$$

$$w_0^T \cdot w(n+1) \geq n\alpha \rightarrow ③$$

if we apply Cauchy - Schwarz inequality

$$\|\vec{w}_0\|^2 \; \|\vec{w}(n+1)\|^2 \geq [\vec{w}_0^T \cdot \vec{w}(n+1)]^2$$

$$\|\vec{w}_0\|^2 \; \|\vec{w}(n+1)\|^2 \geq n^2\alpha^2$$

$$\|\vec{w}(n+1)\|^2 \geq \frac{n^2\alpha^2}{\|\vec{w}_0\|^2} \quad -$$

$$\|\vec{w}(n+1)\|^2 \leq \sum_{k=1}^{n} \|x(k)\|^2$$

$$\|\vec{w}(n+1)\|^2 \leq n\beta$$

$$\frac{\eta^2_{max}\,\alpha^2}{\|w_0\|^2} = \eta_{max}\beta$$

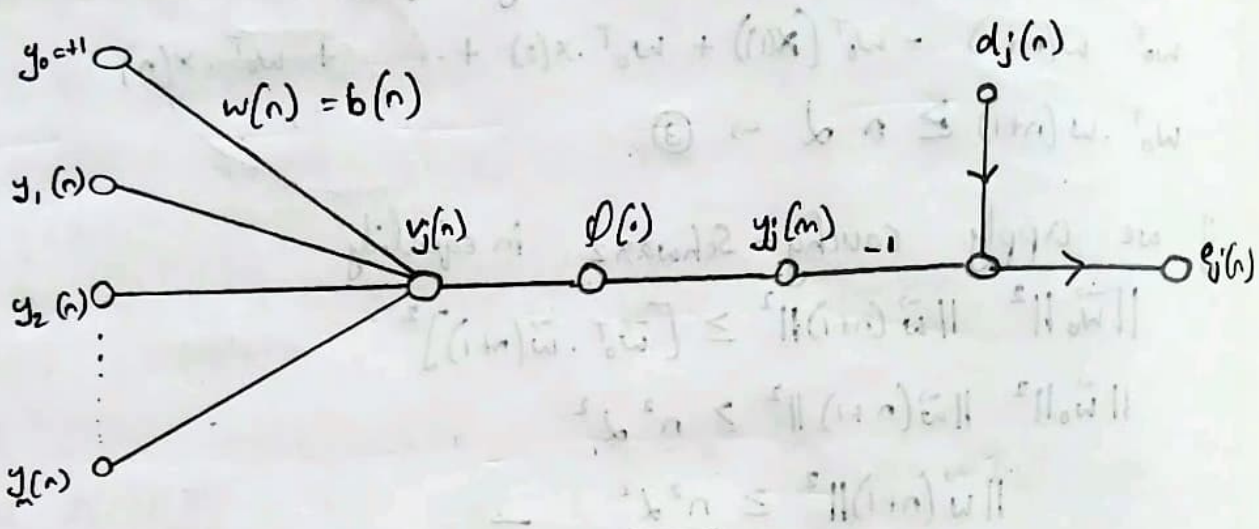
$$\Rightarrow \eta_{max} = \frac{\beta\,\|w_0\|^2}{\alpha^2}$$

Q3. Explain the Back Propagation Algorithm.

Ans: Back Propagation is a supervised learning algorithm used to train multilayer feedforward neural networks.

→ A multilayer feedforward network involves.

- → Input layer
- → One or more hidden layers
- → Output layer.



Steps of Back Propagation Algorithm

Step-01: Initialization:

-> Initially the synaptic weights and thresholds are assigned suitable small random values.

-> when no prior information is available, the weights & thresholds are selected from a uniform distribution whose mean is zero.

-> The variance is selected so that the induced fields lie around the transition region of the sigmoid activation function.

Step-02: Presentation of Training Examples.

->A training example consists of:- $\{x(n), d(n)\}$,

where $x(n)$ -> input vector

$d(n)$ -> desired response vector.

-> The training examples are presented to the network one after another.

-> For each example, forward and backward computations are performed

Step-03: Forward Computation

→ In the forward pass, the input is applied to the input layer.
→ The computation proceeds layer by layer from the input towards the output.
→ For neuron j in layer l, the induced local field is calculated as:

$$v_j^{(l)}(n) = \sum_i w_{ij}^{(l)}(n)\, y_i^{(l-1)}(n)$$

→ The output of the neuron is then obtained using the activation function:

$$y_j^{(l)}(n) = \phi_j(v_j^{(l)}(n))$$

Step-04: Calculation of Error:

→ At the output layer, the actual output is compared with the desired output.
→ The error signal for output neuron j is $e_j(n) = d_j(n) - y_j(n)$
→ The instantaneous error energy of the network is calculated from the error signals.

$$e(n) = \frac{1}{2} \sum_j e_j^2(n)$$

→ The error provides the information required to modify the synaptic weights.

Step-05: Backward Computation:

→ The error is propagated backwards from the output layer towards the hidden layers.
→ The algorithm calculates the local gradient for each neuron.

$$w_{ji} \rightarrow v_j \rightarrow y_j \rightarrow e_j \rightarrow \mathcal{E}$$

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

→ Error function: $E(n) = \frac{1}{2} e_j^2(n) \Rightarrow \frac{\partial \mathcal{E}}{\partial e_j} = e_j(n)$

→ Error definition: $e_j(n) = d_j(n) - y_j(n) \Rightarrow \frac{\partial e_j}{\partial y_j(n)} = -1$

→ Neuron output:

$$y_j(n) = \phi(v_j(n)) \Rightarrow \frac{\partial y_j}{\partial v_j(n)} = \phi_j'(v_j(n))$$

→ Induced local field:

$$v_j(n) = \sum_i w_{ji}(n) y_i(n) \Rightarrow \frac{\partial v_j}{\partial w_{ij}} = y_i(n)$$

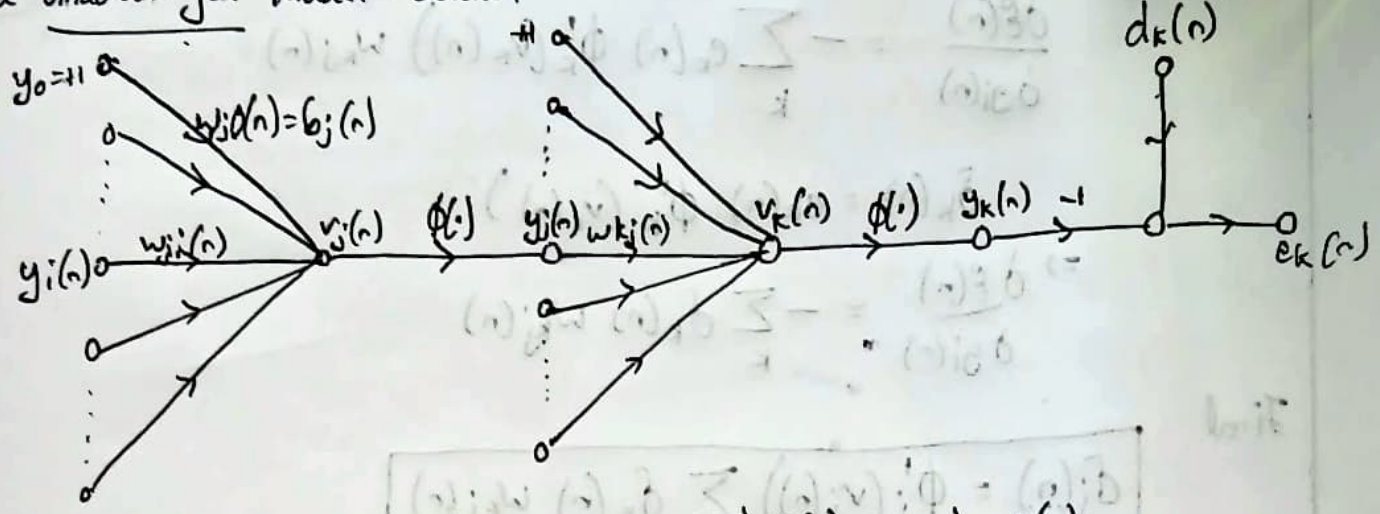
$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = e_j(n)(-1)\,\phi_j'(v_j(n))\,y_i(n)$$

$v_j \rightarrow y_j \rightarrow e_j \rightarrow \mathcal{E}$

$$\delta_j(n) = \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)}$$

$$\boxed{\delta_j(n) = e_j(n)\,\phi_j'(v_j(n))}$$

* Local Gradient for Hidden Neurons:



→ local gradient, $\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} \Rightarrow -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)}$

since $y_j(n) = \phi_j(v_j(n))$ $\quad \frac{\partial y_j(n)}{\partial v_j(n)} = \phi_j'(v_j(n))$

Expand $\frac{\partial \mathcal{E}(n)}{\partial y_j(n)}$, $\quad \mathcal{E}(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n)$

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j(n)}$$

Expand $\frac{\partial e_k(n)}{\partial y_j(n)}$, $\quad e_k(n) = d_k(n) - y_k(n) = d_k(n) - \phi_k(v_k(n))$

$$\frac{\partial e_k(n)}{\partial v_k(n)} = -\phi_k'(v_k(n))$$

Expan $\frac{\partial v_k(n)}{\partial y_j(n)}$, $\quad v_k(n) = \sum_j w_{kj}(n) y_j(n)$

$$\frac{\partial v_k(n)}{\partial y_j(n)} = w_{kj}(n)$$

Expand $\frac{\partial e_k(n)}{\partial y_j(n)} = \frac{\partial e_k(n)}{\partial v_k(n)} \cdot \frac{\partial v_k(n)}{\partial y_j(n)} = -\phi_k'(v_k(n)) \cdot w_{kj}(n)$

Substitute $$\frac{\partial E(n)}{\partial y_j(n)} = -\sum_k e_k(n)\, \phi_k'(v_k(n))\, w_{kj}(n)$$

$$\delta_k(n) = e_k(n)\, \phi_k'(v_k(n))$$

$$\Rightarrow \frac{\partial E(n)}{\partial y_j(n)} = -\sum_k \delta_k(n)\, w_{kj}(n)$$

Final

$$\boxed{\delta_j(n) = \phi_j'(v_j(n)) \sum_k \delta_k(n)\, w_{kj}(n)}$$

Step-06: Weight Adjustment:

After calculating the local gradient synaptic weights are updated using gradient descent.

$$\Delta w_{ji}(n) = -\eta \left[-e_j(n)\, \phi_j'(v_j(n))\, y_i(n) \right]$$

$$\Delta w_{ji}(n) = \eta\, e_j(n)\, \phi_j'(v_j(n))\, y_i(n)$$

$$\boxed{\Delta w_{ji}(n) = \eta\, \delta_j(n)\, y_i(n)}$$

Step-07: Iteration:

- After updating the weights
  - -> The next training example is presented
  - -> Forward computation is performed.
  - -> Error is calculated
  - -> Error is propagated backwards
  - -> Weights are adjusted again.